

```

#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 99999999;

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = (i == j) ? 0 : INF;
    cout << "Enter edges (Source Destination Weight):\n";
    cout << "(Negative weights OK, but no negative cycles)\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v) {
            adj[u - 1][v - 1] = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

void printMatrix(int mat[MAX][MAX], int n, string label) {
    cout << "\n" << label << ":\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (mat[i][j] >= INF) cout << "INF\t";
            else cout << mat[i][j] << "\t";
        }
        cout << endl;
    }
}

// Floyd-Warshall - all pairs shortest path
// dist[][] = shortest distance matrix ("D" matrix from class notes)
// next[][] = path reconstruction matrix ("P" matrix from class notes)
void floydWarshall(int adj[MAX][MAX], int n) {
    int dist[MAX][MAX];
    int next[MAX][MAX];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = adj[i][j];
            if (i != j && adj[i][j] != INF) next[i][j] = j;
            else next[i][j] = -1;
        }
    }
}

```

```

// try every node k as an intermediate point
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF &&
                dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
                next[i][j] = next[i][k]; // path goes through k
            }
        }
    }
}

// negative cycle check: dist[i][i] < 0 means a negative cycle
bool hasNegCycle = false;
for (int i = 0; i < n; i++)
    if (dist[i][i] < 0) { hasNegCycle = true; break; }

if (hasNegCycle) {
    cout << "\nGraph contains a negative weight cycle!\n";
    return;
}

printMatrix(dist, n, "Shortest Distance Matrix (D)");

cout << "\nEnter source and destination (1-indexed) for path: ";
int src, dest;
cin >> src >> dest;
src--; dest--;

if (dist[src][dest] >= INF) {
    cout << "No path exists.\n";
    return;
}

cout << "Shortest distance = " << dist[src][dest] << "\n";
cout << "Path: ";
int at = src;
cout << at + 1;
while (at != dest) {
    at = next[at][dest];
    cout << " -> " << at + 1;
}
cout << endl;
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    floydWarshall(adj, n);
    return 0;
}

```

WHEN TO USE FLOYD-WARSHALL (Scenario Triggers)

- ▶ Question asks for shortest path between ALL PAIRS of nodes, not just one source
- ▶ Question gives a matrix of direct costs and asks for the FULL shortest-distance matrix
- ▶ "Find cheapest route between EVERY pair of cities" style scenarios
- ▶ Graph is SMALL (n usually $\leq$ a few hundred) — Floyd-Warshall is $O(n^3)$, too slow for large graphs
- ▶ Question mentions intermediate nodes / "via node k " reasoning
- ▶ Need to detect a negative cycle AND get all-pairs distances in one pass

Floyd-Warshall vs Dijkstra's/Bellman-Ford — Which One?

- ▶ Need ONE source to all others? → Dijkstra's (no negatives) or Bellman-Ford (negatives OK) — both faster
- ▶ Need ALL pairs (every node to every other node)? → Floyd-Warshall
- ▶ Could also run Dijkstra's/Bellman-Ford from EVERY node individually — but Floyd-Warshall is simpler to code (3 nested loops) for small graphs
- ▶ Floyd-Warshall handles negative edges fine (like Bellman-Ford), just not negative cycles

MANUAL TRACE METHOD (Matrix-by-Matrix, from Class Notes)

Use this exact process — matches the class notes' $D_0, D_1, D_2 \dots D_n$ matrix sequence:

- ▶ 1. Build D_0 = the initial adjacency matrix (INF/ α where no direct edge exists, 0 on diagonal).
- ▶ 2. For each intermediate node k (starting $k=1$), build the NEXT matrix D_k from D_{k-1} .
- ▶ 3. For every cell (i,j) in D_k : check if going THROUGH k is shorter — $D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j])$.
- ▶ 4. Circle/mark any cell that got updated (this shows a shorter path was found via k).
- ▶ 5. Repeat for all $k = 1, 2, \dots n$. The FINAL matrix D_n is the answer — all-pairs shortest distances.
- ▶ 6. For PATH reconstruction (P matrix): whenever you update $D_k[i][j]$ via k , also update $P[i][j] = k$ (or $P[i][j] = P[i][k]$, depending on convention).

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Loop order MATTERS — k must be the OUTERMOST loop. Putting i or j outermost breaks correctness (you'd be using "not yet fully updated" intermediate values).

Diagonal check for negative cycles — if any $\text{dist}[i][i]$ becomes negative after the full algorithm, a negative cycle exists somewhere in the graph.

The P (path) matrix is a SEPARATE structure from the D (distance) matrix — many students only compute D and forget P is often asked for too.

Self-loops: $\text{dist}[i][i]$ should start at 0, not INF — this is the 'cost of staying at the same node'.

$O(n^3)$ time complexity — if asked to justify why this ISN'T used for huge graphs, cite this cubic blowup directly.

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (directed, 4 nodes: 1,2,3,4): $1 \rightarrow 3 = -2$, $2 \rightarrow 1 = 4$, $2 \rightarrow 3 = 3$, $3 \rightarrow 4 = 2$, $4 \rightarrow 2 = -1$

Step-by-step (matches classic CLRS 4-node example):

- ▶ D_0 (initial): direct edges only, INF elsewhere, 0 on diagonal
- ▶ After considering $k=1$: paths through node 1 checked — e.g. $2 \rightarrow 1 \rightarrow 3 = 4 + (-2) = 2$, shorter than $2 \rightarrow 3 = 3 \rightarrow$ update $D[2][3] = 2$
- ▶ After considering $k=2$: paths through node 2 checked — $4 \rightarrow 2 \rightarrow 1 = -1 + 4 = 3$, and $4 \rightarrow 2 \rightarrow 3 = -1 + 2 = 1 \rightarrow$ updates ripple
- ▶ After considering $k=3$: paths through node 3 — $1 \rightarrow 3 \rightarrow 4 = -2 + 2 = 0$, shorter routes emerge
- ▶ After considering $k=4$: final matrix D_4 = complete all-pairs shortest distances

Final Answer (verified against compiled code): $\text{dist}[2][3] = 2$, shortest path from 2 to 3 is $2 \rightarrow 1 \rightarrow 3$ (going via node 1 is cheaper than the direct edge $2 \rightarrow 3 = 3$, because $2 \rightarrow 1 = 4$ and $1 \rightarrow 3 = -2$, totaling 2)

Full D matrix (rows/cols = nodes 1,2,3,4): $\text{row1} = [0, -1, -2, 0]$, $\text{row2} = [4, 0, 2, 4]$, $\text{row3} = [5, 1, 0, 2]$, $\text{row4} = [3, -1, 1, 0]$

EXAMPLE 2 — Negative Cycle Detection

Graph: $A \rightarrow B = 1$, $B \rightarrow C = -3$, $C \rightarrow A = 1$ (same cycle from Bellman-Ford example: total weight = -1)

After running Floyd-Warshall fully, check the diagonal: $\text{dist}[A][A]$ will be negative (since $A \rightarrow B \rightarrow C \rightarrow A$ loops back with total -1).

Exam expects you to say: "Since $\text{dist}[A][A] < 0$ after the algorithm completes, a negative weight cycle exists involving node A. All-pairs distances through this cycle are undefined."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A logistics company operates between 5 warehouses and wants a complete table of cheapest shipping costs between every pair of warehouses, to answer customer queries instantly without recomputation."

How to apply Floyd-Warshall: warehouses = nodes (small n , so $O(n^3)$ is fine), direct shipping costs = edges. Running Floyd-Warshall ONCE precomputes the FULL distance matrix — any future "cheapest cost from warehouse X to Y" query becomes an instant $O(1)$ lookup, which is exactly why all-pairs precomputation is valuable here versus running Dijkstra's repeatedly per query.

EXAMPLE 4 — Path Reconstruction Twist (Medium/Hard style)

Scenario: exam gives the final D matrix and P matrix, and asks you to RECONSTRUCT the actual path from node 2 to node 4 (not just the distance).

Correct handling: start at $P[2][4]$ to find the next node on the path from 2. Keep following $P[\text{current}][4]$ until you reach node 4 itself. Class notes show this exact reconstruction: e.g. " $2 \rightarrow 1 \rightarrow 3 \rightarrow 4$ " built by repeatedly consulting the P matrix, not just reading the distance value.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Floyd-Warshall answer, check:

- Is k the OUTERMOST loop in my trace/code (not i or j)?
- Did I initialize $\text{dist}[i][i] = 0$ and all non-edges to INF correctly?
- Did I check the diagonal for negative cycles after completing all k ?
- If asked for a PATH (not just distance), did I use the P matrix, not just D ?
- Did I justify why Floyd-Warshall ($O(n^3)$) is only used for SMALL graphs, if asked?